Decorators in Python

A decorator wraps a function to add behaviour without editing it: `@dec` means `f = dec(f)`.

How a decorator works

```
def shout(fn):
    def wrapper(*args, **kwargs):
        # extra behaviour
        return fn(*args, **kwargs).upper()
    return wrapper

# same as: greet = shout(greet)
@shout
def greet(name):
    return f"hello, {name}"

print(greet("ann"))
```

```
OUTPUT
HELLO, ANN
```

Built from two ideas: functions are **objects**, and inner functions are **closures** that remember `fn`.

Always add functools.wraps

```
from functools import wraps

def shout(fn):
    # keeps __name__, __doc__, signature
    @wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs).upper()
    return wrapper

@shout
def greet(name):
    """Say hello."""
    return f"hello, {name}"

print(greet.__name__, "-", greet.__doc__)
```

```
OUTPUT
greet - Say hello.
```

Decorators that take arguments

```
# 1: receives the arguments
def repeat(times):
    # 2: receives the function
    def decorator(fn):
        @wraps(fn)
        # 3: runs on every call
        def wrapper(*a, **kw):
            for _ in range(times):
                result = fn(*a, **kw)
            return result
        return wrapper
    return decorator

@repeat(3)
def hi():
    print("hi")

hi()
```

```
OUTPUT
hi
hi
hi
```

Stacking: applied bottom → top

```
def bold(f):  return lambda: f"<b>{f()}</b>"
def ital(f):  return lambda: f"<i>{f()}</i>"

@bold
@ital
def text(): return "hi"

print(text())               # bold(ital(text))
```

```
OUTPUT
<b><i>hi</i></b>
```

Recipe

- Take the function → define `wrapper(*args, **kwargs)` inside.
- Do work **before** / **after** calling `fn(*args, **kwargs)`.
- **Return** the result, decorate `wrapper` with `@wraps(fn)`, return `wrapper`.

Built-in / stdlib decorators	For
`@staticmethod` `@classmethod` `@property`	methods
`@functools.lru_cache` `@cache`	memoization
`@dataclass`	generated class boilerplate
`@contextmanager`	generator → context manager

PITFALL Forgetting `return fn(...)` makes the function return `None`. Forgetting `@wraps` loses the name and docstring. The decorator itself runs at **definition** time; `wrapper` runs at **call** time.